

COS 214 Practical 3: EventFlow

Task 1: Event Concept and Completed Architecture (45 marks)

Team members: Chitambala Sikazwe (u25173384), Gary Mull (u25071247), Louwrens Johansen (u25607414)

1.1 Event Concept: City Lights Festival

City Lights Festival is a single-day, multi-stage outdoor music and community festival held on a large open showground. The festival is deliberately large and physically spread out. Attendees move between a main performance area, a river-front leisure area and a family-oriented area, and each of these behaves as a semi-independent operational zone while still reporting into one central event. This physical spread, combined with the fact that conditions on the ground (weather, capacity, schedule, safety) change throughout the day and need to reach the right operational units quickly, is the situation EventFlow is built to coordinate: a genuine part-whole containment tree (Composite) layered with a genuine one-to-many notification network (Observer).

The showground is organised as a root Venue containing three Zones, one of which itself contains a nested Precinct. This gives the tree the three levels of Composite nesting required below the root. Concretely:

- Venue (root): City Lights Festival, the whole showground for the day.
- Zone: Main Stage Zone, the headline performance area.
- Zone: River Zone, a leisure zone along the river that contains a nested Precinct.
- Zone: Family Zone, a lower-intensity area for younger attendees and workshops.
- Precinct (nested inside River Zone): Food Court Precinct, a cluster of vendors treated as one sub-group.

Venue, Zone and Precinct are three conceptually different kinds of grouping/area, all implemented by the single EventGroup (Composite) class. This is the point of Composite: the same class recurses to any depth without the client needing a different type per level.

Within these groups sit seven concrete kinds of operational unit (Leaves), each with meaningfully different behaviour and not just a different printed name:

- MainStage: hosts performances, pauses and resumes shows, capacity is its audience footprint.
- FrontGate: controls attendee admission, can stop admitting without affecting anyone already inside.
- FoodVendor: serves food from a stall, can suspend kitchen service independently of the surrounding precinct.
- MedicalTeam: provides first aid, and deliberately stays active through most alerts that close other units.
- ShuttleStop: runs a transport route between zones, can change or suspend its route.
- WorkshopTent: runs hands-on activities in the Family Zone, pauses a session rather than evacuating outright.

- InformationDesk: answers attendee questions and relays official announcements as-is.

On top of this containment tree, EventControl acts as the festival's control centre. It is not itself a node in the Composite tree, it is the outside coordinator that raises a Notice (OPEN, CLOSE, SCHEDULE_CHANGE, CAPACITY_ALERT, WEATHER_ALERT, PAUSE, RESUME or EVACUATE) and lets that notice cascade down through the zones that registered interest, reaching only the units that actually need to know, without EventControl ever knowing those units' concrete types.

1.2 GoF Participant Mapping (Observer and Composite)

The table below lists every GoF role required by the practical and states which EventFlow class fills it. Two classes, EventGroup and, in a smaller way, the leaf units, appear more than once because they take part in more than one collaboration. Each row explains the specific collaboration that role belongs to so no relationship is double counted.

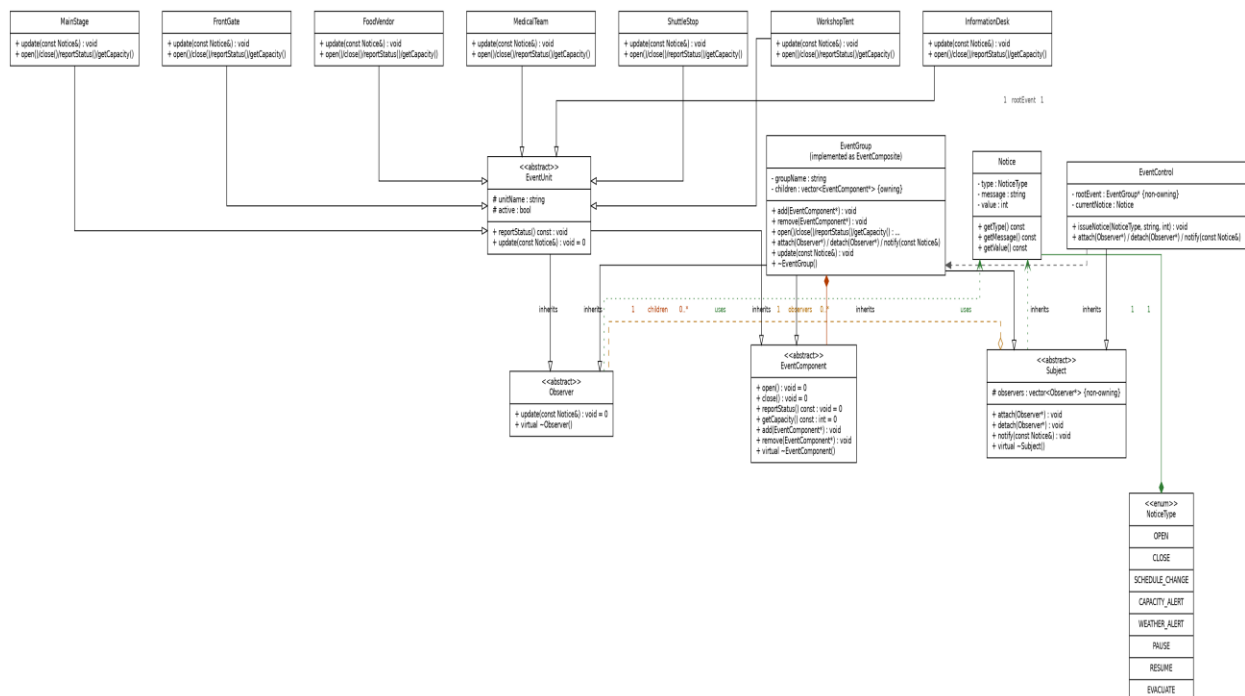
Pattern	GoF Role	EventFlow Class(es)	Why it plays this role
Composite	Component	EventComponent	Declares open(), close(), reportStatus(), getCapacity() and default (throwing) add()/remove() so the client can treat leaves and groups the same way.
Composite	Leaf	MainStage, FrontGate, FoodVendor, MedicalTeam, ShuttleStop, WorkshopTent, InformationDesk	Concrete operational units with no children. Each one implements the four common operations with genuinely different behaviour.
Composite	Composite	EventGroup (implemented as EventComposite)	Owns a vector of EventComponent* and forwards open()/close()/reportStatus()/getCapacity() recursively to its children.
Composite	Client	EventControl / main.cpp	Builds the tree and calls operations through the EventComponent interface without knowing which concrete type it is holding.
Observer	Subject	Subject	Declares attach(), detach() and notify(const Notice&), and owns the (non-owning) list of registered observers.
Observer	ConcreteSubject	EventControl (root issuer) and EventGroup (mid-tree relay)	EventControl issues the first notice for an incident. An EventGroup that has already reacted becomes a subject in its own right so it can pass the notice on to the units below it.

Pattern	GoF Role	EventFlow Class(es)	Why it plays this role
Observer	Observer	Observer	Declares update(const Notice&), the single entry point every interested party has to implement.
Observer	ConcreteObserver	EventGroup (relative to its parent) and every EventUnit leaf	An EventGroup registers with the subject above it. Every leaf registers with the EventGroup that contains it, and reacts to the same notice in its own way.

EventGroup is the class worth pausing on. Within the Composite collaboration it is purely a Composite (it owns and recurses over children). Within the Observer collaboration it plays two separate roles depending on which edge of the tree is being discussed: ConcreteObserver when receiving a notice from whatever is above it, and ConcreteSubject when re-broadcasting that notice to whichever of its own children registered for it. These are two different collaborations between two different pairs of objects, not one pattern doing double duty (expanded on in 1.4(e)).

1.3 Completed UML Class Diagram

The diagram below completes the architecture that Figure 1 of the brief left open. EventComponent and the leaf classes belong to the Composite side, Subject/Observer/EventControl belong to the Observer side, EventGroup sits at the intersection of both since it takes part in both patterns, and Notice/NoticeType is the shared data the two collaborations pass around. Solid diamonds are ownership (composition), hollow diamonds and dashed lines are non-owning association/aggregation, and open triangles are inheritance.



Multiplicities: one EventGroup owns 0..* EventComponent children (composition, filled diamond), one Subject tracks 0..* Observer registrations without owning them (aggregation, hollow diamond), and one EventControl coordinates exactly one root EventGroup. Every abstract base with a virtual member (EventComponent, Subject, Observer) declares a virtual destructor, and EventGroup, which inherits from all three, correctly runs all three destructor chains when a subtree is deleted.

1.4 Design Rationale

(a) Why Composite is a genuine part-whole tree. A FrontGate or a FoodVendor has no meaningful existence outside the Zone or Precinct that physically contains it on the showground. It is not just associated with its group, it is part of it, and deleting the group must delete everything inside it. EventGroup expresses this with real ownership: children are stored as pointers whose lifetime it controls, and ~EventGroup() deletes every child exactly once via the virtual ~EventComponent() destructor chain (see 2.4). The four common operations (open, close, reportStatus, getCapacity) are also whole-part operations by nature, since a Zone's capacity genuinely is the sum of its parts' capacities, which is the strongest sign that Composite, and not a flat list with manual iteration, is the right structure.

(b) Why Observer is required rather than direct hard-coded calls. EventControl cannot know at compile time which units exist in a given festival build, or which of them care about a WEATHER_ALERT versus a SCHEDULE_CHANGE. MedicalTeam ignores weather but reacts to EVACUATE, while ShuttleStop does the opposite. Hard-coding something like "if MainStage then pause, if FoodVendor then suspend" inside EventControl would break the practical's own rule against type-checking dispatch, and would force EventControl to be recompiled every time a new unit type is added. Observer decouples "a notice happened" from "who cares and how they react": units attach and detach at runtime (Task 4 reorganisation, Task 8 demo), and EventControl's notify() loop never needs to change.

(c) Who owns event components. Ownership of the Composite tree is single and exclusive: whichever EventGroup currently holds a child in its children vector owns it, and only that EventGroup may delete it. When a unit is transferred between groups (Task 4.2), the old parent's remove() must run to completion (erasing the pointer from its vector, but not deleting the object) before the new parent's add() takes the same pointer. Ownership moves in one step from one group to the other and is never shared, which is what stops a double free.

(d) Who owns, or does not own, observer references. Subject stores Observer* as plain, non-owning pointers. Attaching a unit as an observer is registering interest, not taking responsibility for its lifetime. The object that owns an EventUnit or EventGroup in the Composite sense (its parent EventGroup) is the only thing allowed to destroy it. To keep those two facts consistent, every Observer's destructor is responsible for detaching itself from whatever Subject it last registered with before it is destroyed, so a Composite deleting its children never leaves a dangling pointer behind in some Subject's observer list further up the tree (implemented and demonstrated in Task 3).

(e) Why a class that is both Subject and Observer is not a misuse of either pattern. GoF patterns describe a collaboration between roles, not a permanent label on a class. EventGroup's Observer role exists in its collaboration with whatever sits above it (it is being told something), and its Subject role exists in a completely separate collaboration with whatever sits below it (it is telling others something). These are two different one-to-many

relationships with two different sets of participants, and EventGroup never plays both roles on the same edge of the tree at once, since a notice always flows strictly downward through a chain of otherwise normal Observer/Subject pairs. The dual role is exactly what the brief's own scenario asks for, that "some event areas receive notifications from above and then notify interested units below."